Serverless Trace Analyzer
Team MAL

Trace Reconstruction and Revisioned Storage

Design Specification
Version 1.1 October 28 2026

Larry Nguyen
Larry.t.nguyen@sjsu.edu

Version History
Introduction
References
Requirements
Functional Overview
    Configuration and External Interfaces
    Debug
        Logging
        Counters
Implementation
Testing
    General Approach
    Unit Tests
Appendix

Version History

Version	Changes
0.1 October 14 2026	Designed the trace schema and come up with general logic of reconstruction, deduplication, and other cases that may come up
1.0 October 20 2026	Wrote the first technical draft covering span grouping, duplicates and conflicts, trace states, revision checks, and the test plan. Used the October 20 merged implementation as the reference.
1.1 October 26 2026	Finished the layout review, improved wrapping for long test names, and checked the worked example, template sections, and internal links.
1.2 October 28 2026	Started to get it working with the sample microservice applications and added critical path estimates

Introduction

My part of this project is taking individual spans and putting them back together into a trace. Spans can arrive out of order, be delivered more than once, or show up after processing has finished. This design explains how I handle those cases, keep track of changes to a trace, and store results that match its latest revision. A revision is the version of the trace after new information changes it.

This work is in trace_schema/schema.sql and the topology.py, store.py, and worker.py files under reconstruction/trace_reconstruction. It was merged in commit 4a40ec3 on October 20 [R1]. The team notes also identify trace schema and reconstruction as my contribution [R2]. This spec covers the shared span format at the reconstruction input, the reconstruction logic, and storage. The adapters, upload and replay tools, AWS setup, dashboard, and service-ranking algorithms are separate parts of the project.

Workbook 2 already gives an overview of the packages, span fields, trace lifecycle, and database. Here, I go into the rules behind that overview: how duplicate records are compared, when the trace state changes, how the database rejects outdated results, and what each test should check. The late-arrival example in the Appendix walks through those rules step by step [R3].

References

[R1] Team MAL repository, commit 4a40ec3e162cddf725dd507820695005ee857e66, Trace schema and reconstruction, October 20 2026. This spec describes my implementation at that commit. The Appendix lists the relevant files.

[R2] meetings/2026-09-30-team.txt and meetings/2026-10-20-team.txt. Trace schema assignment and merge of Larry's schema and reconstruction contribution.

[R3] Project Workbook 2 - Team MAL.docx, local copy dated October 22 2026. Chapter 6, Trace reconstruction and storage design; Chapter 7, Test strategy and Test definitions.

[R4] trace_schema/verification.txt. Results recorded for the October 13 database and contract checks and the October 14 ingestion checks. These are earlier checks, separate from the unit tests rerun on October 26.

Requirements

These requirements add detail to Workbook E4 and N4. I keep a trace's processing state separate from its topology, which describes how the spans connect. Settling a trace means the worker has stopped waiting for now. It does not mean every span has arrived.

ID	Requirement and expected result
TR1	Use (run_id, trace_id, span_id) to identify a span. The same source IDs in different runs must stay separate, so replaying a dataset does not overwrite an earlier run.
TR2	Allow a child to arrive before its parent. The same accepted records must produce the same parent and child maps regardless of arrival order, including the order of children.

ID	Requirement and expected result
TR3	Treat a retry as DUPLICATE when only received_at_unix_ns or source.raw_reference changes. Keep the accepted record and revision, and do not restart the quiet window.
TR4	Keep the first accepted record when another record with the same identity has different content. The first conflict sets has_conflicts, increases the revision once, and reopens a settled trace. Repeated conflicts do not keep increasing it.
TR5	Settle a trace only when it is OPEN, the selected revision is still current, and the required deadline or end-of-input condition is met. Use QUIET_WINDOW, HARD_TIMEOUT, or END_OF_INPUT as the reason.
TR6	When a new span arrives after settlement or timeout, reopen the trace, increase its revision, clear the settlement fields, and hide results from older revisions in the current view.
TR7	Label topology INVALID, UNKNOWN, INCOMPLETE, or CONNECTED using the rules below. Keep missing parents, inferred relationships, conflicts, and comparable out-of-bounds intervals in the quality information.
TR8	Save an analysis only if its revision is still current and the trace is no longer OPEN. Roll back an outdated write and raise StaleTraceError so a later poll can retry.
TR9	Validate canonical records before storing them. If validation or a database write fails in ingest_many, roll back the whole batch. Keep missing timing and unavailable features explicit instead of setting them to zero.

Functional Overview

The adapter first writes spans in the shared canonical format for an existing run. PostgresStore validates the records and sends them to trace_v1.ingest_span. PostgreSQL locks the trace row while changing it, so two deliveries cannot change the same trace at the same time. ReconstructionWorker settles traces that are due, then reconstructs settled revisions that do not yet have a result for its algorithm version. The later analysis stages read the accepted spans, their quality information, and the results for the current revision.

How incoming spans are handled

Result	What changes in storage
INSERTED	Store the new span and increase revision by one. Set the trace to OPEN, clear the settlement fields, and update last_changed_at and last_received_at. Reset opened_at only if the trace was previously settled or timed out.
DUPLICATE	Keep the accepted span. Update only last_received_at. The state, revision, opened_at, and last_changed_at stay the same, so a retry does not move the deadlines.
CONFLICT	Keep the accepted span. The first conflict sets has_conflicts, increases the revision, reopens the trace, clears settlement fields, and updates last_changed_at. Later conflicts update last_received_at without another revision change.

Settlement rules

A trace can settle after a quiet window, at the hard timeout, or when input is explicitly finished. The quiet deadline is last_changed_at + quiet_window_ms. The hard deadline is opened_at + max_open_ms. If both are due, the SQL store chooses HARD_TIMEOUT. Finishing a run sets ingestion_runs.ended_at and uses END_OF_INPUT for its open traces. A timeout says the worker stopped waiting; the topology still has to be checked separately. In this implementation, a new late span can reopen a trace even after the run was marked finished.

The defaults are a 5000 ms quiet window and a 60000 ms maximum open period. I use last_changed_at for the quiet window because repeated deliveries should not keep a trace open. Only new information resets that clock. I also allow a missing parent during ingestion, since the child may simply have arrived first. A parent foreign key would reject that normal delivery order.

Configuration and External Interfaces

Setting or dependency	Configuration and behavior
Canonical schema	Schema version 1.0.0. Check the full JSON Schema and cross-field rules before SQL ingestion. Each record must use the same identity_kind as its run.
PostgreSQL	Use the trace_v1 schema from trace_schema/schema.sql and psycopg for the connection. PostgreSQL 14+ is the intended minimum. Apply this schema to a fresh database; it is not a migration for an existing one.

Setting or dependency	Configuration and behavior
Run configuration	quiet_window_ms must be greater than zero, and max_open_ms must be at least quiet_window_ms. create_run stores the application, environment, source format, dataset, and configuration. Calling it again with the same run ID does not update that configuration.
Worker options	poll_once defaults to 100 candidates in each phase. Local run_forever polls every 1 second. The algorithm version for this implementation is topology-v1. Polling must continue when input stops so the last traces can settle.
Input	The CLI reads a canonical JSON object, JSON array, or JSONL file. inspect groups records in memory. Database ingestion uses the full validator. The topology function expects records that have already been validated and deduplicated.
Output	ReconstructedTrace returns parent and child maps with quality information. trace_analyses stores input_revision, algorithm_version, configuration, topology, and span_features. Code reading the current view must choose the algorithm version it needs.

For the CLI, put --dsn before the command. The integration environment supplies the database connection and scheduling. S3, SQS, Lambda, credentials, and dashboard endpoints are configured separately.

Debug

When debugging a trace, I would start with its run and trace IDs, revision, state, settlement_reason, and has_conflicts flag. The quality fields then explain what is missing or uncertain. Comparing last_received_at with last_changed_at helps separate retry traffic from new spans. If a current result disappears, check whether a late span reopened the trace before assuming the result was lost.

Logging

The CLI prints JSON counts for INSERTED, DUPLICATE, and CONFLICT. A poll or finish prints WorkerStats, and inspect prints each trace's topology and quality. Validation errors identify invalid fields, while an outdated analysis write raises StaleTraceError. This version does not have centralized logging or a stored audit entry for every incoming record.

Conflict handling also needs support from the code that delivers records. ingest_span keeps the accepted record and a trace-level conflict flag, then returns CONFLICT. The caller is responsible for saving the rejected payload or sending it to quarantine with its raw archive reference. The CLI currently reports counts only. A delivery wrapper can log run_id, trace_id, revision, and the result without changing how duplicates are compared.

Counters

The ingestion counts show how many records in a batch were INSERTED, DUPLICATE, or CONFLICT. WorkerStats reports settled, analyzed, and stale for one worker call. stale includes a failed settlement, a changed revision, or an outdated analysis write. These counts are not stored totals for the trace. Since the database does not keep retry records, its quality output uses duplicate_count=null and duplicate_count_available=false. TraceGroup can count duplicates seen during its own in-memory grouping.

Implementation

Data structures and trace grouping

Structure	Fields and purpose
TraceGroup	Holds run_id, trace_id, records indexed by span_id, duplicate_count, and conflict_span_ids. add makes a deep copy of the first accepted record so later changes by the caller cannot change it.
TraceCandidate	A frozen record with run_id, trace_id, revision, reason, and has_conflicts. It records what the worker selected, but that revision may change before processing finishes.
ReconstructedTrace	A frozen result with run_id, trace_id, topology, root_span_id, sorted spans, parent and child maps, and quality. span_features converts each span's parent and ordered child IDs into a form that can be stored.
WorkerStats	Counts settled, analyzed, and stale traces for the current worker call.

To compare retries, semantic_content makes a deep copy and removes only received_at_unix_ns and source.raw_reference. Everything else still counts, including adapter_version, timing, status, and source information. Python compares JSON with sorted object keys, and PostgreSQL compares the matching JSONB content. Array order still matters. The SQL comparison removes the two arrival fields with this expression:

```
(p - 'received_at_unix_ns') #- '{source,raw_reference}'
```

This duplicate check handles repeated delivery of the same observation. The Alibaba adapter separately combines caller and callee records before they reach reconstruction. If another record changes meaningful content under an accepted span ID, it is a conflict. The reconstruction code keeps the original record rather than editing it in place.

Building the topology

reconstruct_trace first rejects empty input, records from different run/trace pairs, and repeated span IDs. It builds a map by span ID and sorts spans by (start is absent, integer start or zero, span_id). That puts timed spans first and uses span_id to break ties. It then finds declared roots and missing parents,

counts UNKNOWN and INFERRED relationships, and follows parent chains to check for cycles. Child lists use the same sort order, so changing arrival order does not change the result.

Priority	When the label applies
1 INVALID	There is a parent cycle or more than one declared ROOT.
2 UNKNOWN	There is no INVALID condition, but at least one parent relationship is UNKNOWN.
3 INCOMPLETE	The first two rules do not apply, but a parent is missing or there is not exactly one root.
4 CONNECTED	There is one root, with no missing parents, unknown relationships, or cycles. INFERRED edges are allowed, but their count is still reported.

OTLP links do not become parent edges. If a child extends outside its parent's interval and the two intervals can be compared, the code increases out_of_bounds_child_count while keeping the topology label. It does not fill in unknown time domains, missing intervals, or inferred relationships as if they were measured. root_span_id is set only when exactly one ROOT is present, even if other quality issues limit later analysis.

Grouping and cycle checks use O(n) extra memory for n spans, and sorting takes O(n log n). Each trace is loaded into memory. The worker's poll limit caps the number of traces selected, not the number of spans inside one trace. Limits for very large traces still need to be tested before production use.

Database writes and revision checks

ingest_span creates a trace row if needed and locks it with SELECT FOR UPDATE. It then checks the span identity and updates the stored content and trace state in the same transaction. The combined span primary key prevents two accepted rows with the same identity. ingest_many commits after the whole batch succeeds. A validation or SQL error rolls the batch back, so the caller must retry the complete batch.

settle_trace checks the run, trace, expected revision, and OPEN state again before making a change. The deadline must be due, or ended_at must be set for END_OF_INPUT. If a check fails, it returns false. Selecting a trace for settlement does not bypass these database checks.

Before saving an analysis, the worker fetches the records and checks that the revision still matches the one it selected. The guard_trace_analysis database trigger then locks the trace row and checks again. It rejects a different input_revision or an OPEN trace. This second check handles a span arriving after the Python check. Older analysis rows stay in history, but current_trace_analyses shows only rows for the current revision of a non-OPEN trace.

There is still a concurrency limit to keep in mind. Two workers can save analyses for the same revision because analysis_id is unique, but (run_id, trace_id, input_revision, algorithm_version) is not. The current view may return more than one result or algorithm version. The reader needs to choose the required version and one result consistently, such as the newest computed_at followed by analysis_id to break ties. Duplicate span delivery is handled safely, but analysis publication is not guaranteed to happen exactly once.

Worker steps and recovery

```
poll_once(limit=100)
  for candidate in due_traces(limit):
    count settle success or stale result
  for candidate in pending_analyses(version, limit):
    fetch records and revision
    skip if selected revision changed
    reconstruct and attempt guarded publication
    count analyzed or stale result
```

If the worker stops after settlement but before saving an analysis, the trace remains settled without a result. The next pending_analyses query can select it again. A failed analysis write rolls back. If a new span arrives after a result was saved, the trace gets a new revision and the old result disappears from the current view until the new revision is processed. Other SQL and validation errors are passed back to the caller. The worker itself does not provide a retry queue.

pending_analyses checks algorithm_version, but it does not compare a configuration hash. If the reconstruction rules or configuration change enough to affect results, use a new algorithm version or define how to rerun existing traces. finish_run also selects pending analyses across runs before filtering to the requested run, with a 100000-candidate limit. A large shared backlog may need more polls.

Implementation scope

I split this work into the canonical schema and validator, grouping and topology logic, PostgreSQL storage and trace state, and the worker and CLI. Each part can be tested separately. In commit 4a40ec3, the saved analysis contains topology and quality information. root_duration_ns and critical_path are null. self_time_ready and critical_path_ready are false with deferred_to_sprint_4 reasons. Timing calculations, bottleneck ranking, tail sampling, and cloud event connections are later work and are not counted as finished features in this spec.

Testing

General Approach

The tests use small canonical examples where the expected trace can be checked by hand. They check the stored records, revision, state, and visible result, along with the parent and child maps. The grouping tests run without AWS or a database. SQL behavior needs a separate test database that can be discarded afterward. Each independent replay should have its own run ID.

The October 26 test run used a clean copy of commit 4a40ec3. All 24 reconstruction tests passed: 16 grouping and topology tests, 3 store tests, 3 worker tests, and 2 Astronomy Shop helper tests. Of the 16 contract checks, 15 passed and the SQL syntax check was skipped because the optional pglast dependency was missing. The Astronomy Shop tests use local fixtures, so they do not show that a live deployment was tested.

The earlier database results in trace_schema/verification.txt were recorded on October 13 with PostgreSQL 18 in an isolated database, and the test data was rolled back [R4]. Those database checks were not rerun during this document revision. The store unit tests use fake connections, so actual locking and deadline behavior still need database testing. The table below separates the earlier SQL checks from additional tests to add.

Functional and database tests

ID and coverage	Procedure and expected result
F1 Recorded SQL check	Send a child before its root, then send the root. Both records should be accepted. After finishing the run and saving the reconstructed result, expect CONNECTED topology with both spans present.
F2 Recorded SQL check	Resend an accepted record with only arrival information changed. Expect DUPLICATE, the same revision, and the same span count. The original quiet deadline must still allow settlement.
F3 Recorded SQL check	Change operation_name for an existing identity. Expect CONFLICT, the original content, and one revision increase. Sending the same conflict again must not increase the revision again.
F4 Recorded SQL check	Try settling before the quiet deadline or with an old revision. Expect false and OPEN state. Then set ended_at and settle the current revision with END_OF_INPUT. Expect SETTLED.
F5 Recorded SQL check	Save a result at revision r, add a new span, and query the current view. Expect OPEN at r+1 with the result from r hidden. After settling and saving r+1, only results for that revision should be visible.
F6 Recorded SQL check	Keep a trace OPEN until its hard deadline, then settle it with HARD_TIMEOUT. Expect TIMED_OUT with that reason. Check the topology separately from the timeout state.
F7 Additional concurrency test	Use two database connections and control their order. A selects a revision, B adds a span, then A tries to settle or publish the old revision. Settlement must fail or the analysis write must roll back. The old result must not appear as current.
F8 Additional recovery test	Settle a trace and stop before saving its analysis. A new worker poll should find that revision still waiting for analysis and save the result without inserting its spans again.
F9 Additional concurrency test	Let two workers publish for the same settled revision at the same time. Both writes may succeed. Check that the reader chooses one result and does not count the trace twice.

For an end-to-end check, send the same canonical batch through reconstruction twice and compare the accepted span IDs and topology. The dashboard and analysis code should also be checked to make sure a reopened trace loses its current result and that quality warnings are respected. AWS delivery, throughput, and dashboard freshness need their own system tests.

Unit Tests

These tests are in reconstruction/tests. The first column gives the method names, and the second explains what each test checks. This keeps the focus on reconstruction instead of repeating the workbook test list for the whole project.

Existing test	**Expected result**
`test_parent_first_and_child_first_produce_same_topology`	Reordering input leaves the parent and child maps unchanged. Input with children first still produces CONNECTED topology.
`test_exact_retry_ignores_arrival_metadata`	Changing reception time and the raw reference returns DUPLICATE. The in-memory duplicate count becomes 1.
`test_semantic_conflict_keeps_first_observation`	A changed operation returns CONFLICT. The accepted operation stays POST /checkout, and the conflicting span ID is reported.
`test_same_trace_ids_in_different_runs_are_isolated`	Two runs using the same source trace IDs stay separate, with three accepted spans in each group.
`test_rejects_mixed_trace_input` `test_rejects_undeduplicated_input`	Records with different trace keys and repeated identities raise ValueError.
`test_connected_trace_and_deterministic_child_order`	One root and two children produce CONNECTED topology with a fixed child order. critical_path_ready stays false.
`test_missing_parent_is_incomplete` `test_no_root_is_incomplete`	The missing-parent example is INCOMPLETE and lists parent ID 0000000000000099. A child without a root reports root_count 0.
`test_multiple_roots_are_invalid` `test_parent_cycle_is_invalid`	Two roots produce INVALID with root_count 2. A parent cycle produces INVALID with cycle=true.
`test_unknown_relationship_has_unknown_topology` `test_inferred_parent_can_be_connected_but_is_reported`	An UNKNOWN root relationship produces UNKNOWN. An INFERRED edge can still be CONNECTED, with inferred_parent_count 1.
`test_out_of_bounds_interval_is_quality_evidence_not_invalid_topology`	A child starting before its parent keeps CONNECTED topology and sets out_of_bounds_child_count to 1.
`test_links_do_not_create_parent_edges` `test_spans_without_start_time_sort_by_span_id_after_timed_spans`	A link does not replace the parent relationship. A span without a start time sorts after a timed root.
`test_ingest_many_reports_database_dispositions`	The fake database returns one of each result, so each count is 1 and the batch commits once.

Existing test	Expected result
`test_validation_failure_rolls_back_batch` `test_stale_analysis_error_rolls_back_and_is_classified`	Invalid input rolls back once without committing. An outdated analysis write rolls back and raises StaleTraceError.
`test_poll_settles_then_publishes_topology`	The worker reports settled=1, analyzed=1, stale=0 and saves CONNECTED topology at revision 3.
`test_stale_revision_is_not_published` `test_finish_run_uses_end_of_input`	A revision 3 candidate cannot publish after revision 4 arrives. finish ends the run, settles the trace, and analyzes it.

Appendix

Example of a late span

This example starts at revision 0 with no conflicts. R is the root, C is its child, and L arrives later. It follows the SQL rules for one trace in one run, rather than showing production results.

Step	Observation or action	Expected stored state
1	C arrives before R.	INSERTED; revision 1; OPEN. No parent row is required.
2	R arrives.	INSERTED; revision 2; OPEN. Topology now has one root and a known parent for C.
3	C is retried with a new raw reference.	DUPLICATE; revision 2; quiet deadline unchanged.
4	Quiet deadline passes; worker settles and publishes.	SETTLED at revision 2; CONNECTED analysis is visible.
5	Previously unseen L arrives.	INSERTED; revision 3; OPEN; old revision 2 analysis is hidden.
6	A worker tries to publish its old revision 2 result.	Guard rejects the write; transaction rolls back as stale.
7	Quiet deadline passes again.	SETTLED at revision 3; replacement analysis includes R, C and L.
8	A changed operation arrives under C's existing identity.	CONFLICT; revision 4; OPEN; original C remains; has_conflicts=true. A repeated conflict stays at revision 4.

Local verification commands

Run these commands from the repository root using the existing virtual environment. The first runs the reconstruction tests, and the second checks the schema and contract. Neither needs AWS.

```
Push-Location reconstruction
..\.venv\Scripts\python.exe -m unittest discover -s tests -t .
Pop-Location
.\.venv\Scripts\python.exe trace_schema/check_schema.py
```

The CLI example below needs a disposable database with schema.sql already applied. The parallel-trace fixture uses run 00000000-0000-4000-8000-000000000001. To use a connection other than the default on port 55432, put --dsn before the subcommand.

```
Push-Location reconstruction
..\.venv\Scripts\python.exe -m trace_reconstruction init-run `
  00000000-0000-4000-8000-000000000001 --identity-kind OTEL
..\.venv\Scripts\python.exe -m trace_reconstruction ingest `
  ../trace_schema/examples/parallel-trace.json
..\.venv\Scripts\python.exe -m trace_reconstruction finish `
  00000000-0000-4000-8000-000000000001
Pop-Location
```

Relevant source files

The schema and validator are in trace_schema/canonical-span.schema.json and validate.py. The SQL functions and views are in trace_schema/schema.sql. Under reconstruction/trace_reconstruction, topology.py handles grouping and topology, store.py handles database access, worker.py runs reconstruction, and __main__.py exposes the CLI. The unit tests listed above are in test_topology.py, test_store.py, and test_worker.py under reconstruction/tests. There are also two database check scripts, trace_schema/check_database.py and reconstruction/check_database.py. Follow their instructions for disposable databases because they handle transactions and test data differently.